

# Paradigmi di Programmazione - A.A. 2022-23

Esame Scritto del 6/9/2023

## CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

### Esercizio 1 [Punti 4]

Applicare la  $\beta$ -riduzione alla seguente  $\lambda$ -espressione usando la strategia **call-by-name** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda x.x)((\lambda x.\lambda y.xy)y)$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la  $\beta$ -riduzione (redex) ed evidenziando le eventuali  $\alpha$ -conversioni.

#### SOLUZIONE:

Una possibile soluzione:

$$\begin{aligned} & \frac{(\lambda x.x)((\lambda x.\lambda y.xy)y)}{\rightarrow (\lambda x.\lambda y.xy)y} \\ \equiv_{\alpha} & \frac{(\lambda x.\lambda k.xk)y}{\rightarrow \lambda k.yk} \end{aligned}$$

### Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f g x y = if g x then
  match x with
  | (0,k) -> k
  | (_,k) -> g x
else
  y ;;
```

**SOLUZIONE:**

Struttura del tipo:

$G \rightarrow X \rightarrow Y \rightarrow \text{RIS}$

Uso per convenzione  $G, X, Y$  come variabili di tipo per la variabili  $g, x, y$ ,  $\text{RIS}$  come variabile di tipo del risultato, e  $A, B, C, \dots$  come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

```
G = X -> bool      (da guardia dell'if)
X = int * K         (da pattern matching)
RIS = bool          (da secondo caso p.m.)
K = bool            (da casi del p.m.)
Y = RIS             (da ramo else)
```

Ne consegue:

```
G = (int * bool) -> bool
X = (int * bool)
Y = bool
RIS = bool
```

Tipo inferito:

```
((int * bool) -> bool ) -> (int * bool) -> bool -> bool
```

### Esercizio 3 [Punti 7]

Assumendo il seguente tipo di dato che descrive alberi binari "colorati" di interi i cui nodi intermedi possono avere colore nero o rosso, mentre le foglie sono di colore verde:

```
type colored_tree =
| BlackNode of int * colored_tree * colored_tree
| RedNode of int * colored_tree * colored_tree
| GreenLeaf of int
```

si definiscano, usando i costrutti di programmazione funzionale di OCaml, le funzioni `color_tree_lists` e `color_forest_lists` con tipi

```
color_tree_lists : colored_tree -> int list * int list * int list
color_forest_lists : colored_tree list -> int list * int list * int list
```

tali che:

- `color_tree_lists ct` restituisca una tripla i cui tre elementi sono liste contenenti tutti i valori interi presenti rispettivamente in nodi neri, rossi e verdi dell'albero;
- `color_forest_lists ctlis` esegue lo stesso calcolo di `color_tree_lists`, ma su una lista di alberi colorati, restituendo una unica tripla i cui elementi contengono tutti i valori presenti rispettivamente in nodi neri, rossi e verdi di tutti gli alberi della lista. (Nota: `color_forest_lists` può usare al suo interno `color_tree_lists`.)

L'ordine degli elementi nelle varie liste non è importante.

(SEGUE NEL RETRO)

Ad esempio, dato il seguente albero colorato

```
let ct = BlackNode (10,
  RedNode (5,
    BlackNode (2,
      GreenLeaf 1,
      GreenLeaf 2),
    GreenLeaf 4),
  RedNode (7,
    GreenLeaf 2,
    GreenLeaf 5))
```

abbiamo che `color_tree_lists ct` restituisce `([10; 2], [5; 7], [1; 2; 4; 2; 5])`.

Inoltre, data la seguente lista di alberi colorati:

```
let ctlis = [ BlackNode (1,          (* primo albero *)
  GreenLeaf 2,
  GreenLeaf 3);
  RedNode (4,          (* secondo albero *)
  GreenLeaf 5,
  GreenLeaf 6);
  BlackNode (6,        (* terzo albero *)
  RedNode (7,
    GreenLeaf 8,
    GreenLeaf 9),
  RedNode (10,
    GreenLeaf 11,
    GreenLeaf 12)) ]
```

abbiamo che `color_forest_lists ctlis` restituisce `([1; 6], [4; 7; 10], [2; 3; 5; 6; 8; 9; 11; 12])`.

SOLUZIONE

### SOLUZIONE:

Una possibile soluzione:

```
let rec color_tree_lists ct =
  match ct with
  | BlackNode (v,ct1,ct2) -> let (b1, r1, g1) = color_tree_lists ct1 in
    let (b2, r2, g2) = color_tree_lists ct2 in
      (v::(b1@b2), r1@r2, g1@g2)
  | RedNode (v,ct1,ct2) -> let (b1, r1, g1) = color_tree_lists ct1 in
    let (b2, r2, g2) = color_tree_lists ct2 in
      (b1@b2, v::(r1@r2), g1@g2)
  | GreenLeaf v -> ([],[],[v]);;

let color_forest_lists ctlis =
  let tmp_list = List.map color_lists ctlis in
  let merge (a,b,c) (d,e,f) = (a@d,b@e,c@f) in
  List.fold_left merge ([],[],[]) tmp_list;;
```

## Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con il tipo di dato astratto `IntCollection` per la definizione di collezioni di dati di tipo intero. Il costrutto `Collection(e1,...,eN)` definisce una collezione contenente

gli elementi  $x_1, \dots, x_N$  ottenuti valutando le espressioni  $e_1, \dots, e_N$ . Una collezione può contenere più copie dello stesso valore.

Esempio: `Collection(3+2,7,8-3)` definisce la collezione contenente 5,7,5.

Inoltre, devono essere previste le seguenti operazioni:

- `Union(c1,c2)`, date due collezioni `c1` e `c2` restituisce una nuova collezione con gli elementi di entrambe
- `Exists(p,c)`, dato un predicato `p` (ossia, una funzione che restituisce un valore booleano) e una collezione `c`, restituisce `True` se almeno uno degli elementi della collezione soddisfa il predicato (ossia, se esiste almeno un elemento su cui la funzione predicato restituisca `True`)

Ad esempio (in una sintassi concreta simile a OCaml):

```
let c1 = Collection(1,1+1,4-1) in      (* c1 contiene 1,2,3 *)
let c2 = Collection(2,2*2,2*3) in      (* c2 contiene 2,4,6 *)
let c3 = Union(c1,c2) in               (* c3 contiene 1,2,3,2,4,6 *)
let pred x = (x=3) in                 (* pred restituisce true se x=3 *)
let ris1 = Exists(pred,c1) in          (* ris1 è true *)
let ris2 = Exists(pred,c2) in          (* ris2 è false *)
let ris3 = Exists(pred,c3) in          (* ris3 è true *)
....
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.

### SOLUZIONE:

Una possibile soluzione:

```
type exp = ...
  | IntCollection of exp list
  | Union of exp*exp
  | Exists of exp*exp

type evT = ...
  | CollVal of evT list

let rec eval e s = match e with
  ...
  | IntCollection e1 -> let val_lis = (List.map (fun x -> eval x s) e1) in
                        CollVal val_lis
  | Union (e1,e2) -> (eval e1 s) @ (eval e2 s)
  | Exists (e1,e2) -> (match (eval e1 s) with
                        | Closure (i,body,env) ->
                          let collection = eval e2 s in
                          let check x = (eval body (bind env i x)) = Bool(True)
                          List.exists check collection
                        | _ -> failwith "Error"
                        )
```